support *session affinity*, also known as *sticky sessions*. That is, once an updated version of your app has been deployed, any new sessions will be handled by this latest version, but any existing user sessions will continue to be serviced by the old version. This is important because the old version may be incompatible with your updated version of the app in some way, particularly if routes or database schemas have changed.

You can still achieve zero-downtime deploys with Swarm, but it will involve some extra work, typically requiring you to run a reverse proxy in front of the app that *does* provide session affinity. Zero-downtime deploys can also be achieved with Kubernetes, but this similarly involves some work.[9]

Security

It's beyond the scope of this book to teach you the ins and outs of securing your cloud-based infrastructure, but if you're building a production environment, this will be a key area to get right. Unfortunately, there's no *one-size-fits-all* approach, particularly as things can vary greatly between cloud platforms.

A good starting place is Docker's own docs on the subject.[10] Make sure you see the various pages under "Security" in the menu—there's no "Next" button at the bottom of the page. Key topics are: only using trusted images, scanning images for vulnerabilities, not running containers as root,[11] and locking down firewalls to the bare minimum ports required, plus more involved ways to lock down your Docker installation.

More Advanced Architectural Possibilities

So far we've used Swarm's built-in load-balancing capabilities to distribute incoming requests to different containers backing a given service. However, as you get more experienced, you may want to use more sophisticated setups with things like HAProxy[12] or NGINX[13] to do your own proxying and load balancing.

Not only is this possible, but you can run HAProxy or NGINX instances in containers themselves, building your own images with your config files. You can also use Docker's network primitives[14] to create different network configurations.

9. https://medium.com/@diegomrtnzg/redirect-your-users-to-the-same-pod-by-using-session-affinity-on-kubernetes-baebf6a1733b
10. https://docs.docker.com/engine/security/security/
11. https://docs.docker.com/engine/security/userns-remap/
12. http://www.haproxy.org
13. https://www.nginx.com
14. https://docs.docker.com/v17.09/engine/swarm/networking/